# OWLuedo: детектив на машине вывода

## Пример к лекции 3: определяемые классы, цепочки свойств, замыкание мира на 64 индивидах

О. М. Атаева  А. П. Халов

Осень 2026

# Убийство в особняке Тамануар

- Дом из четырёх этажей и девяти комнат, в комнатах зоны: танцпол, уголок у камина, леса для росписи.
- Семнадцать человек. Каждый где-то был и чем-то владел: всего 66 связей «был в» и 114 связей «владел».
- Тридцать предметов: острые, тупые, мягкие, одно огнестрельное оружие и продукты с плотностью.
- Омар Тир мёртв. Его нашли в мастерской через три минуты после смерти.

Учебный пример OWLuedo из набора Cluedo4KG (IRIT, Тулуза, авторы Н. Эрнандес и К. Прадель, лицензия CC BY-NC 4.0). Лёгкая онтология и три таблицы csv даны, тяжёлую онтологию студент строит по заданию на шесть страниц. Здесь то же задание выполнено в Owlready2 и прогнано Pellet.

Данные: 401 тройка, 64 индивида, 13 классов из таблиц.

**Кто убийца? Чем? Кто невиновен?**

## Главное

Ответ не записан нигде. Он получается из определений классов, цепочек свойств и трёх замыканий мира.

# Что уже есть и что нужно построить

**Дано: лёгкая онтология**

- Классы House, Room, Person, Object.
- Свойства: комната содержит объект, человека, соседняя комната, дом содержит комнату, человек жив.
- Данные из csv: типы предметов, где что лежит, кто где был, кто чем владел, плотность продуктов, кто жив.

**Построить: тяжёлая онтология**

- Иерархия: подвижные и неподвижные объекты, оружие, продукты жидкие и порошковые, места, причины смерти.
- Характеристики свойств: транзитивность «находится в», симметричность соседства, функциональность плотности.
- Определяемые классы: жертва, место преступления, застреленный, убийца.
- Цепочки свойств: был в комнате, значит был на этаже и в доме.
- Замыкание мира, без которого ответа нет.

## Главное

Тот же маршрут, что в задании 3 курса: от вопросов к классам, свойствам, ограничениям, затем машина вывода.

# Классы: иерархия и разбиения

```
class Object(Thing): pass
class MobileObject(Object): pass
class ImmobileObject(Object): pass
class BluntObject(MobileObject): pass
class SharpObject(MobileObject): pass
class Firearm(MobileObject): pass
class Product(MobileObject): pass
class Weapon(MobileObject):
    equivalent_to = [BluntObject | SharpObject | Firearm]
class LiquidProduct(Product): pass
class PowderProduct(Product): pass
Product.equivalent_to.append(LiquidProduct | PowderProduct)
AllDisjoint([LiquidProduct, PowderProduct])

class Person(Thing): pass
class Victim(Person): pass
class Murderer(Person): pass
class Innocent(Person): pass
Person.equivalent_to.append(Victim | Murderer | Innocent)
AllDisjoint([Victim, Murderer, Innocent])
```

- Оружие есть объединение трёх классов: определяемый класс через «или».
- Продукт есть либо жидкость, либо порошок: объединение плюс дизъюнктность, это разбиение.
- Человек есть либо жертва, либо убийца, либо невиновный. Разбиение нужно для последнего шага: кто не жертва и не убийца, тот невиновен.
- Дом не этаж, этаж не комната: две дизъюнктности, которые потом поймают ошибку в данных.

## Главное

Разбиение класса есть объединение подклассов вместе с их попарной дизъюнктностью. Без дизъюнктности вывод «невиновен» невозможен.

# Свойства: характеристики и области

```
class isLocatedIn(ObjectProperty, TransitiveProperty):
    domain = [Place]; range = [Place]
class roomOnFloor(isLocatedIn): ...
class floorInHouse(isLocatedIn): ...
class areaInRoom(isLocatedIn): ...
class hasAsNeighbourRoom(ObjectProperty,
        SymmetricProperty, IrreflexiveProperty): ...
class hasDensity(DataProperty, FunctionalProperty):
    domain = [Product]; range = [float]
class dueTo(ObjectProperty, FunctionalProperty):
    domain = [CauseOfDeath]; range = [MobileObject]
class ownsOrOwned(ObjectProperty): ...
class owns(ownsOrOwned): pass
class isOrWasInPlace(ObjectProperty): ...
class isInPlace(isOrWasInPlace): pass
class personInRoom(isInPlace): range = [Room]
```

- «Комната на этаже», «этаж в доме», «зона в комнате» суть частные случаи одного транзитивного свойства «находится в».
- Соседство симметрично и иррефлексивно: комната не соседка самой себе.
- У продукта одна плотность, у причины смерти один предмет: функциональность.
- «Владеет» есть частный случай «владел или владеет», «находится» частный случай «был или находится».

## Главное

Иерархия свойств работает как иерархия классов: факт про частный случай есть факт про общее свойство.

# Определяемые классы: жертва, место преступления, убийца

```
Victim.equivalent_to.append(Person & isAlive.value(False))
Victim.is_a.append(hasCauseOfDeath.exactly(1, CauseOfDeath))
GunshotInjury.is_a.append(dueTo.some(Firearm))
class CrimeScene(Room): equivalent_to = [Room & roomContainsPerson.some(Victim)]
class ShotVictim(Victim):
    equivalent_to = [Victim & discoveredAfter.some(ConstrainedDatatype(int, max_exclusive=5))]
ShotVictim.is_a.append(hasCauseOfDeath.some(GunshotInjury))
PowderProduct.equivalent_to.append(Product & hasDensity.some(ConstrainedDatatype(float, min_exclusive=2.5)))
Murderer.equivalent_to.append(Person
    & ownsOrOwned.some(responsibleFor.some(causeOf.some(Victim)))   # владел предметом, вызвавшим смерть
    & isOrWasInPlace.some(CrimeScene))                               # и был на месте преступления
```

- Жертва есть человек, у которого записано «не жив». Место преступления есть комната, где находится жертва.
- Застреленный есть жертва, найденная раньше чем через пять минут: ограничение на значение данных.
- Порошок есть продукт плотнее 2,5: тоже ограничение на данные, машина вывода сама сортирует продукты.
- Убийца определён через два пути длиной три: от человека к предмету к причине к жертве и от человека к месту преступления.

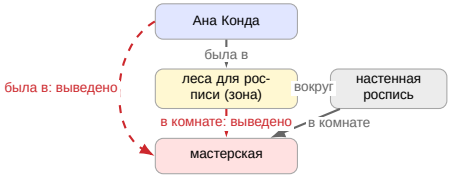
## Главное

Определяемый класс записывает вопрос. Машина вывода наполняет его, когда факты дотягиваются до определения.

# Цепочки свойств: как Ана Конда оказалась в мастерской

```
isOrWasInPlace.property_chain.append(
    PropertyChain([isOrWasInPlace, isLocatedIn]))
roomContainsArea.property_chain.append(
    PropertyChain([roomContainsObject, isCentreOf]))
isInPlace.property_chain.append(
    PropertyChain([owns, objectInRoom]))
```

- Был в месте, место внутри другого: был и там.
- Объект в комнате, зона организована вокруг объекта: зона в комнате.
- Владеет предметом, предмет в комнате: сам в комнате.




В данных Ана Конда в мастерской не записана. Выведено по двум цепочкам: роспись стоит в мастерской, значит леса вокруг росписи тоже в мастерской, значит Ана там была.

## Главное

Две цепочки в обе стороны образуют цикл, который OWL 2 запрещает, Pellet такие аксиомы отбрасывает. Нужна одна.

# Четыре шага: открытый мир и три замыкания

| шаг | что добавлено | что вывела машина (Pellet, 1–3 с) |
|---|---|---|
| 1 | тяжёлая онтология, 64 индивида | жертва Omar_Ty, место преступления Atelier, застреленный Omar_Ty. Убийцы: никого |
| 2 | единственная причина смерти: `CauseOfDeath` ≡ {theCause} | theCause есть огнестрельное ранение, Omar_Ty hasCauseOfDeath theCause. Чем именно: неизвестно |
| 3 | единственное оружие: `Firearm` ≡ {Revolver_Python_357} | theCause dueTo Revolver. Убийца: Ana_Conda, она владела револьвером и была в мастерской. Omar_Ty killedBy Ana_Conda |
| 4 | `Murderer` ≡ {Ana_Conda}, `Victim` ≡ {Omar_Ty}, все люди различны | пятнадцать невиновных |

- Шаг 1 воспроизводит вывод задания: несмотря на все аксиомы, убийцу назвать нельзя. В доме могут быть ещё причины смерти и ещё оружие.
- Шаг 4 отвечает на вопрос задания «как заключить, что остальные невиновны»: без различности индивидов любой мог бы оказаться Аной Кондой под другим именем.

## Главное

Каждое замыкание записывает «больше ничего нет» для одного класса. Ответ появляется ровно тогда, когда замкнуто то, что нужно вопросу.

# Ошибка в данных и объяснение

Задание предлагает добавить два факта: оранжерея есть этаж, и оранжерея содержит комнату-оранжерею. Если записать их про один и тот же индивид:

```
ПРОТИВОРЕЧИЕ: Explanation(s):
1)   Greenhouse_Floor type Room
     Floor disjointWith Room
     Greenhouse_Floor type Floor
```

- Дизъюнктность этажа и комнаты, объявленная при построении, срабатывает на данных.
- Объяснение называет три аксиомы, из которых следует противоречие. Решение: два индивида, этаж-оранжерея и комната-оранжерея, так и сделано в данных авторов.
- Та же ситуация, что строка журнала с организацией в поле рецензента в примере редактора.

## Главное

Дизъюнктность пишется ради таких ошибок: без неё индивид спокойно был бы и этажом, и комнатой.

## Что показывает пример

1. Разбиение класса (объединение плюс дизъюнктность) нужно, чтобы выводить «не то, значит это»: невиновен тот, кто не жертва и не убийца.
2. Иерархия свойств и транзитивность дают выводы о местах: был в комнате, значит был на этаже и в доме.
3. Ограничения на значения данных (плотность, минуты до обнаружения) классифицируют индивидов без правил.
4. Цепочки свойств соединяют связи в новые: именно они приводят убийцу на место преступления. Цепочки не должны образовывать цикл.
5. Замыкание мира делается по одному классу за шаг, и каждый шаг даёт ровно один новый вывод.
6. Ответ про невиновных требует различности индивидов: открытый мир допускает, что два имени называют одного человека.

### Главное

Пример в 64 индивида проверяет всё, что было в лекции 3: определяемые классы, иерархию, цепочки, объяснения и цену открытого мира.